Character-level vs Word-level Language Modeling

An Empirical Comparison of RNNs and Transformers

Course: MEL29 - Language Resources and Processes

Author: Yue Ma

GitHub: https://github.com/nikk909/MEL29_Language_Resources_and_Processes

Abstract

Sequence models are central to modern natural language processing. Two major families are recurrent neural networks (RNNs), which update a hidden state step by step, and Transformers, which use self-attention to relate positions in a sequence (Vaswani et al., 2017). This paper reports a small but complete experiment that trains and compares four language models in PyTorch: CharRNN, CharTransformer, WordRNN, and WordTransformer. The training corpus is English Wikipedia text of about 55,000 characters. We record cross-entropy loss and perplexity (PPL) across eight epochs and inspect text generated from a fixed prompt, including a temperature sweep for the word-level Transformer. At word level, the Transformer clearly outperforms the RNN (final PPL about 38.8 versus 66.3). At character level, the simpler RNN is better (about 4.9 versus 10.8). Absolute character-level scores look lower mainly because the vocabulary is much smaller, so they should not be read as proof that character models are stronger overall. Generated text remains weak under this data and training budget. Overall, the results partly support the claim that self-attention scales better for sequence generation, but only under specific conditions: word-level modeling benefits, while character-level modeling under the same shallow setup does not.

Keywords: RNN, Transformer, self-attention, perplexity, character-level, word-level

1 Introduction

Language is sequential: the next character or word depends on what came before. A language model learns this conditional structure and can be used for generation, scoring, and many downstream tasks. For years, RNNs and their gated variants (LSTM, GRU) were the default tools for sequence modeling. After 2017, Transformer architectures based on self-attention became the main backbone of large language models (Vaswani et al., 2017). The practical question is not only which name is newer, but how the two designs organize memory, use compute, and handle long context—and whether those differences still matter in a small, fully controlled experiment.

Historically, language modeling moved from sparse n-gram counts to neural predictors that share parameters across similar contexts (Bengio et al., 2003). Recurrent networks compressed history into a state vector; attention later offered dynamic reading of past positions (Bahdanau et al., 2015); Transformers made attention the main block. This shift is widely described as a scalability win, yet most claims come from large-scale settings. A transparent side-by-side run that keeps data, objective, and budget fixed while changing only architecture and token granularity remains useful for checking whether those design differences still matter at small scale.

This study runs that comparison. We build character-level and word-level pipelines and train four models: CharRNN, CharTransformer, WordRNN, and WordTransformer. The corpus is English Wikipedia text of roughly 55,000 characters. Training lasts eight epochs on CPU.

Evaluation uses loss, perplexity, fixed-prompt generation, and a temperature sweep for Word-Transformer. The goal is reproducible empirical evidence, not only a restatement of famous papers.

The central thesis is that self-attention is, in principle, easier to scale for sequence generation because it can connect any two positions directly and can parallelize over sequence length during training. However, this advantage is conditional. On small data, short training, and shallow models, a simple RNN may still win on local tasks such as character prediction. Therefore, claims about scalability should be checked within the same tokenization granularity and should report both supporting and opposing results.

The research questions are as follows.

1. How do loss and PPL evolve for the four models over training?
2. When is it valid to compare absolute scores across character-level and word-level settings?
3. What do fixed-prompt generations and temperature sampling show about coherence and diversity?

The remainder of the paper is organized as follows. Section 2 reviews RNN recurrence, vanishing gradients, LSTM/GRU, Bahdanau attention, self-attention, positional encoding, causal masking, and character versus word bias. Section 3 describes research type, data, models, and evaluation. Section 4 analyzes metrics, curves, and samples. Section 5 answers the research questions and outlines limitations. Code and figures: `CharVSWordGPT_YueMa.ipynb`, `results/pic`, `results/data`.

2 Theory: State of Research and Academic Debate

2.1 From n-grams to neural language models

Classical n-gram models estimate the next-token distribution from a fixed window of previous tokens. They are simple and historically successful, but they cut off context outside the window and suffer from sparsity as the order n grows. Smoothing and backoff help, yet discrete counts do not share statistical strength across similar words or similar contexts.

Neural language models replace sparse counts with continuous representations. Bengio et al. (2003) showed that learned word vectors plus a neural predictor can generalize to unseen sequences by exploiting semantic similarity. Two words that never co-occur with the same successor can still benefit each other if their embeddings are close. Two answers later dominated practice: compress history into a recurrent hidden state, or retrieve past positions with attention. The rest of this section compares those answers and derives expectations for the present experiment.

2.2 RNN recurrence as compressed memory

A basic RNN updates a hidden state h_t from the previous state and the current input:

$$h_t = f(W h_{t-1} + U x_t + b). \tag{1}$$

The hidden state acts as a summary of the past. Parameters are shared across time, and the update rule is online: after seeing token t, the model has a new state ready for token $t+1$. For short sequences and modest vocabularies, this is often enough.

The weaknesses are structural rather than accidental:

1. Sequential bottleneck: computing h_t requires h_{t-1}, so training does not fully exploit GPU parallelism over time. Throughput grows poorly with sequence length compared with architectures that allow parallel token mixing.
2. Memory wall: a fixed-size vector must carry all history; distant information is easily overwritten.
3. Inference latency: generation remains step-wise because the state must be updated at every step.

These limits become more serious as sequences grow and as the required dependency span exceeds what a compressed state can reliably store. Much of the later literature on gated RNNs and attention can be read as attempts to soften this memory wall without abandoning sequence modeling.

2.3 Vanishing gradients, LSTM, and GRU

Training RNNs with backpropagation through time multiplies Jacobians across steps. If those factors are repeatedly smaller than one, gradients vanish; if larger than one, they may explode. Bengio et al. (1994) analyzed why learning long-term dependencies with gradient descent is difficult: distant signals may exist in the forward pass, but useful gradient may fail to reach early parameters. Truncated BPTT further shortens the credit-assignment window by design.

LSTM (Hochreiter and Schmidhuber, 1997) addresses the gradient-flow problem with a

cell state and gates (input, forget, output). The additive cell path lets information travel across many steps with fewer repeated squashing nonlinearities. GRU (Cho et al., 2014) simplifies the design with update and reset gates and often matches LSTM quality with fewer parameters. Empirically, gated RNNs made sequence systems far more practical than vanilla RNNs.

Critically, gates improve trainability *inside* the recurrent paradigm. They do *not* remove sequential dependence during training, and they do *not* give constant-path access between arbitrary positions. When data volume, model size, and context length increase, efficiency and long-range modeling again become bottlenecks—one reason attention moved from an add-on to a core architecture. In this paper the baseline is a simpler multi-layer RNN: gating would likely help the recurrent side, but it would not automatically erase self-attention's path-length and parallelism advantages.

2.4 Bahdanau attention and the move to self-attention

Before Transformers, attention entered mainstream NLP mainly through neural machine translation. Bahdanau et al. (2015) argued that forcing the decoder to rely on a single fixed-length vector from the encoder was a bottleneck. Their solution lets the decoder soft-align over encoder states at each step: a query is compared with encoder keys, and a weighted context vector conditions the next output. This is dynamic reading of relevant history.

Three points remain central. First, attention weights provide soft alignment and mark a shift from opaque state compression to addressable memory. Second, Bahdanau attention uses a feed-forward scoring function; later Transformer work preferred scaled dot-product attention for

efficiency, but the functional goal is the same. Third, Bahdanau et al. (2015) still used RNNs for both encoder and decoder: attention was an interface, not a replacement for recurrence. After 2015, the debate asked whether recurrence remained necessary once attention could retrieve distant information; Vaswani et al. (2017) answered that question aggressively.

2.5 Self-attention, Transformers, positional encoding, and causal masking

Self-attention applies the retrieval idea inside one sequence. Each position produces queries, keys, and values; similarity scores decide how much to read from other positions. Scaled dot-product attention is:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{2}$$

The scaling term $\sqrt{d_k}$ keeps dot products from growing too large with dimension. Multi-head attention runs this in several subspaces so different heads can specialize. Compared with RNN state passing, attention shortens the path between distant tokens: interaction can happen in one layer rather than through many recurrent steps. This is the central theoretical argument for scalability.

Vaswani et al. (2017) proposed the Transformer and argued that attention can replace recurrence as the main sequence engine. For autoregressive language modeling, causal masking is essential: when predicting position i, the model must not see tokens after i. Without correct masking, future leakage can inflate training metrics. Without recurrence, positional encoding (sinusoidal or learned) supplies order information to content-based attention.

In theory, self-attention can out-scale recurrence because training can parallelize over sequence length under the mask, any two tokens can interact with a short path, and multi-head structure offers flexible relational capacity. The costs are that standard attention is often $O(n^2)$, and small-data settings may undertrain attention models relative to compact RNNs with stronger local bias. This paper tests a shallow RNN against a shallow causal Transformer at two granularities and treats disagreement as theoretically informative rather than as experimental failure.

2.6 Character-level vs word-level inductive bias

Tokenization changes the learning problem before any architecture is chosen.

- Character-level: small vocabulary, long sequences, strong local spelling structure. Much of the immediate loss comes from local letter and punctuation regularities.
- Word-level: larger vocabulary, shorter sequences, weaker spelling pressure, more rare-word and `<UNK>` issues. Absolute perplexity is usually higher because the softmax chooses among more classes.

A low character-level PPL is not automatically "better modeling of language" than a higher word-level PPL, because the prediction spaces differ. Fair architecture claims should be made *within* one granularity.

2.7 Synthesis and expectations for this experiment

The literature supports a conditional claim rather than a slogan. Recurrent models remain competitive when dependencies are short and data are limited. Attention, from Bahdanau et al. (2015) to Vaswani et al. (2017), shortens interaction paths and enables parallel token mixing, which is why self-attention is widely judged more scalable—yet that does not guarantee victory in every shallow, short-epoch run. We therefore expect: (1) at word level, Transformer PPL should often beat RNN PPL; (2) at character level with little data, a simple RNN may match or beat a shallow Transformer; (3) character absolute PPL looks better mainly because of vocabulary size; (4) generation stays limited at $\sim$55k characters and 8 epochs; (5) temperature mainly changes diversity. Section 4 tests these expectations.

3 Methodology

3.1 Research type and design

This study is a controlled comparative experiment. We keep the corpus, optimizer family, epoch count, and next-token objective fixed, and we vary architecture (RNN vs. Transformer) and tokenization (character vs. word). The aim is internal comparison under one budget, not state-of-the-art pretraining.

Regarding reliability, the pipeline is implemented in one notebook with a fixed random seed (42), so reruns under the same software stack should be close. Shared preprocessing, shared batch construction logic, and exported JSON metrics reduce hidden degrees of freedom. We

did not repeat multiple seeds, so uncertainty from seed variance is not quantified.

Regarding validity, construct validity relies on loss and perplexity for distribution fit and on qualitative generation for readability; low PPL does not imply fluent prose. Internal validity is helped by holding data and epoch count constant across models. External validity is limited: English encyclopedia text does not represent dialogue, code, or multilingual settings. Conclusions about "scalability" are therefore scoped to this design.

3.2 Data source and preprocessing

English Wikipedia text is fetched through the public MediaWiki API (`prop=extracts&explaintext=1`), cleaned by removing section heading markers and compressing whitespace, and concatenated to 55,047 characters. The small size supports transparent CPU reproduction and clear generation failure modes.

3.3 Tokenization and batching

In the character pipeline, each unique character maps to an integer index (vocabulary size 90). In the word pipeline, text is split into words and punctuation; types with frequency below 2 map to `<UNK>` (vocabulary size 1,011; $\sim$10,323 tokens). For batches, input x predicts right-shifted y; character window length is 100, word window length is 40, and batch size is 32. Comparisons remain within each granularity.

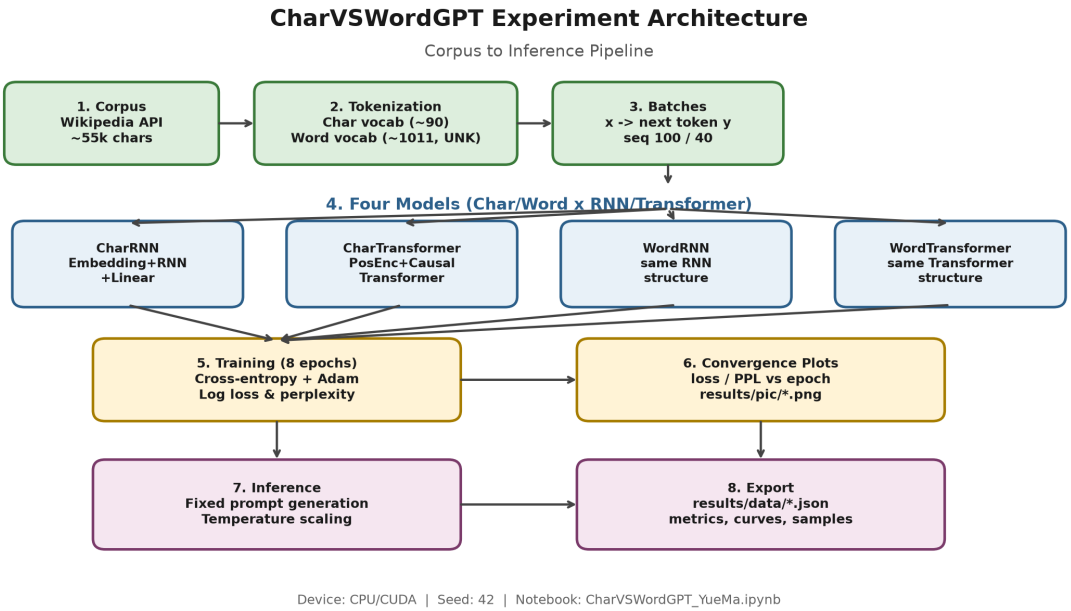

Figure 1: Experimental architecture roadmap: character-level and word-level pipelines, each with an RNN and a causal Transformer branch under a shared corpus and budget.

Table 1: Model architectures used in the experiment.

Model	Granularity	Structure
CharRNN	character	Embedding + multi-layer RNN + Linear
CharTransformer	character	Embedding + learned positional encoding + causal TransformerEncoder + L
WordRNN	word	same RNN structure
WordTransformer	word	same Transformer structure

3.4 Model architectures

As illustrated in Figure 1 and Table 1, the study compares four models under matched shallow settings (e.g., two layers, embedding/d_model $\approx$ 128). RNN training detaches hidden states across batches (truncated BPTT) and uses gradient clipping. The Transformer uses causal masking (`is_causal` / subsequent mask) so that position i cannot attend to future positions. The RNN baseline is a standard multi-layer RNN rather than a full LSTM/GRU stack; Section 2.3 implies that gated variants would likely strengthen the recurrent side.

3.5 Training procedure and evaluation metrics

Shared hyperparameters: 8 epochs, Adam learning rate 0.003, seed 42, CPU. For each epoch we store mean cross-entropy and $\mathrm{PPL} = \exp(\overline{\mathrm{CE}})$. Generation uses the fixed prompt `"Germany is a"`; WordTransformer is also sampled at temperatures 0.2, 0.7, 1.0, 1.5, and 2.0. Artifacts go to `results/pic/` and `results/data/`. Analysis compares within-granularity PPL, discusses cross-granularity scores only with the vocabulary caveat, reads samples for surface coherence, and uses temperature to separate competence from decoding randomness.

4 Analysis & Discussion

4.1 Final metrics and convergence

Table 2 and Figures 2–3 show three patterns. At word level, WordTransformer PPL (38.79) is about 41% lower than WordRNN (66.32), consistent with shorter interaction paths under

Table 2: Final loss/PPL (8 epochs) and PPL at epochs 1 and 8.

Model	Final Loss	Final PPL	PPL@1	PPL@8
CharRNN	1.5905	4.91	19.86	4.91
CharTransformer	2.3765	10.77	21.95	10.77
WordRNN	4.1945	66.32	314.56	66.32
WordTransformer	3.6581	38.79	311.40	38.79

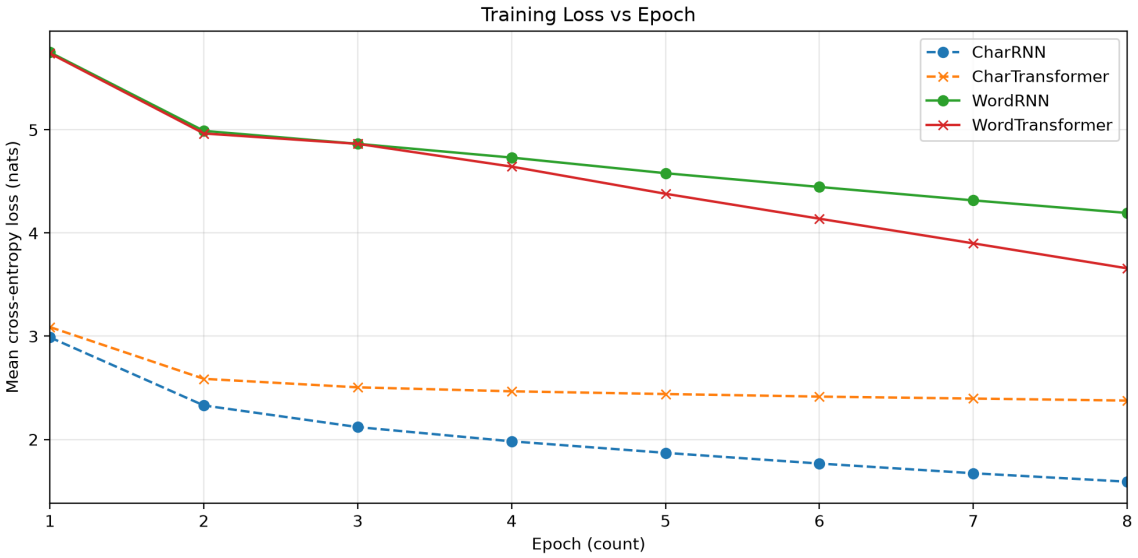

Figure 2: Training loss versus epoch. X-axis: epoch count (1–8). Y-axis: mean cross-entropy loss (nats).

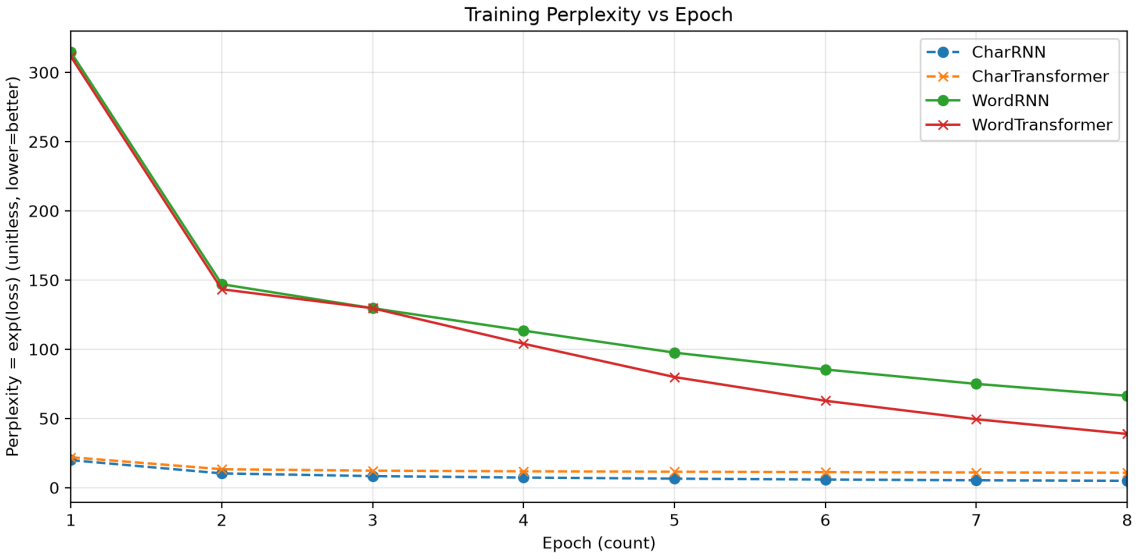

Figure 3: Training perplexity versus epoch. X-axis: epoch count (1–8). Y-axis: perplexity (unitless, lower is better).

self-attention (Sections 2.4–2.5). At character level, CharRNN (4.91) beats CharTransformer (10.77); local spelling favors a compact RNN, and the attention model appears under-optimized in eight epochs—the small-data caveat in Sections 2.5 and 2.7. Across granularities, absolute character PPL looks much lower mainly because there are $\sim$90 vs. $\sim$1,000 classes (Section 2.6).

All curves decline. CharRNN falls quickly and stays best among character models; CharTransformer improves early then flattens; word-level models start above PPL 300, then WordTransformer separates clearly from WordRNN. Local character structure may stress long-term recurrent credit assignment less than word-level lexical choice (Section 2.3), helping explain the character-level reversal.

4.2 Generation quality and temperature scaling

Fixed prompt: `"Germany is a"` (from `generation_samples.json`).

- CharRNN: letter strings that sometimes look word-like but are often invalid.
- CharTransformer: spelling is also broken and hard to read.
- WordRNN: mostly real word forms, but many `<UNK>` tokens and rapid loss of syntax.
- WordTransformer: corpus-related words (e.g., Rhineland, European) yet remains incoherent.

Lower training PPL means better fit to the next-token distribution, not fluent writing. With $\sim$55k characters and 8 epochs, the experiment is enough for comparative curves, not for a usable generator. Character models fail mainly at orthography; word models fail at syntax and topical continuity—consistent with Section 2.6. For WordTransformer, low temperature (0.2) is

conservative and repeats <UNK>; mid temperatures show more topical words but still broken sentences; high temperatures (1.5–2.0) look more random. Temperature changes diversity, not competence; architecture comparisons should keep decoding settings fixed.

4.3 Answering the research questions and limitations

1. Training dynamics: all models learn; winners are WordTransformer (word) and CharRNN (character). The word-level gap widens over epochs; the character-level ranking stabilizes early.
2. Comparability: absolute cross-granularity PPL is misleading; within-granularity comparison is the valid architecture test.
3. Generation and temperature: samples remain weak; temperature mainly trades repetition against chaos.

The thesis from Section 1 is only *partly* supported, matching Section 2.7. Limits include single seed, no per-model LR search, shallow RNN (not LSTM/GRU), small English Wikipedia text, and qualitative generation only. The claim is scoped to matched shallow settings, not a universal ranking of all sequence architectures.

5 Conclusions

This paper compared RNNs and Transformers for character- and word-level language modeling on a $\sim$55k-character Wikipedia corpus. Findings: WordTransformer beats WordRNN in PPL; CharRNN beats CharTransformer; lower absolute character scores largely reflect vocab-

ulary size; generation stays poor.

Winners depend on granularity; cross-granularity absolute PPL should not be over-interpreted; generation and temperature show limited fluency.

Self-attention's scalability advantage is real but conditional—supported here at word level, not at character level.

Practically, within-granularity metrics should be reported, and causal masking and positional encoding should be treated as first-class design choices.

Future work can increase data and epochs, add LSTM/GRU baselines, deepen the Transformer, use multiple seeds, measure longer-context throughput, and add stronger generation metrics. Until then, the safest conclusion remains conditional: self-attention helps here at word level and does not help at character level under the same budget.

References

Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1409.0473

Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A neural probabilistic language model. *Journal of Machine Learning Research, 3*, 1137–1155. https://www.jmlr.org/papers/v3/bengio03a.html

Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks, 5*(2), 157–166. https://doi.org/10.1109/72.279181

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1724–1734). https://doi.org/10.3115/v1/D14-1179

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30* (pp. 5998–6008). https://arxiv.org/abs/1706.03762

Appendix A List of Figures and Tables

List of Figures.

List of Tables.

Appendix B Result paths and reproduction

Exported experimental artifacts:

Table 3: Result file paths.

Type	Path
Architecture roadmap	`results/pic/architecture_roadmap.png`
Loss curve (Figure 2)	`results/pic/fig_training_loss.png`
Perplexity curve (Figure 3)	`results/pic/fig_training_perplexity.png`
Final metrics	`results/data/final_metrics.json`
Per-epoch curves	`results/data/training_curves.json`
Generation samples	`results/data/generation_samples.json`

Reproduction notes:

- Run the experiment notebook: `CharVSWordGPT_YueMa.ipynb`
- Fixed protocol: seed 42; 8 epochs; Adam learning rate 0.003; character seq_len 100; word seq_len 40; batch size 32
- Install dependencies from `requirements.txt` in the repository root